CodeAlpha

Learn | Build | Grow

Project Report

Books Web Scraping and Data Analysis

Extracting book information from a website, cleaning the data, and performing analysis with visualizations



Submitted By	:	Dhivyabharathi M
Department	:	B.Tech (AI&DS)
College	:	Peri Institute of Technology
Organization	:	CodeAlpha
Submission Date	:	30/09/2026

Thank you, CodeAlpha!

TABULATION:

S.No	Table of content	Page No
1	Cover Page	1
2	Tabulation	2
3	Introduction	3
4	Objectives	4
5	Tools and Technologies	5
6	Methodology	6-7
7	Python Implementation	8-22
8	Dataset and Data Cleaning	23-33
9	Data Analysis and Charts	34-44
10	Results and Conclusion	45-47
11	Dataset (CSV Data)	48
12	References	49

INTRODUCTION:

Web scraping is a data collection technique used to extract information from websites using programming languages such as Python. It helps collect structured data from web pages and transform it into useful datasets for data analysis and decision-making.

In this project, Book Web Scraping is performed using Python libraries such as Requests and BeautifulSoup. The Books to Scrape website is used as a practice source to collect information about books, including their titles, prices, availability, and ratings. The extracted data is organized into a structured dataset using the Pandas library.

The main purpose of this task is to understand website HTML structure, identify relevant information, extract data accurately, and create a custom dataset for analysis. The collected data is cleaned, processed, and visualized using Python tools such as Matplotlib.

This project provides practical knowledge of web scraping, data preprocessing, and exploratory data analysis. It also demonstrates how raw web data can be converted into meaningful information that supports analytical tasks and real-world applications.

Objectives – Book Web Scraping and Data Analysis:

Main Objective:

To collect book-related information from a public practice website using Python web scraping techniques, create a structured dataset, and perform data analysis to extract meaningful insights from the collected data.

Specific Objectives:

- Data Collection: To extract book titles, prices, availability, and ratings from the Books to Scrape website using Python.
- Web Scraping: To learn and apply Python libraries such as BeautifulSoup and Requests for extracting information from website HTML structures.
- Dataset Creation: To organize the collected book information into a structured dataset using Pandas and save it in CSV format.
- Data Cleaning: To clean and preprocess the collected data by removing unwanted characters, handling missing values, and converting data into suitable formats.
- Data Analysis: To calculate book price statistics and analyze rating distributions using Python.
- Data Visualization: To create charts and graphs using Matplotlib to represent the collected data clearly.
- Practical Learning: To develop practical knowledge of web scraping, data preprocessing, and exploratory data analysis for real-world applications

TOOLS AND TECHNOLOGIES:

The following tools and technologies are used in the Book Web Scraping and Data Analysis project:

Tool / Technology	Purpose
Python	Programming and data processing
Requests	Fetching webpage content
BeautifulSoup	Parsing HTML and extracting data
Pandas	Data cleaning and analysis
Matplotlib	Data visualization
Google Colab	Python development and execution
CSV	Storing the collected dataset
Books to Scrape	Practice website and data source

These tools support the complete workflow of web scraping, dataset creation, data preprocessing, and visualization.

METHODOLOGY:

The methodology of this project follows a systematic approach to collect book information from a practice website, create a structured dataset, and perform data analysis using Python libraries.

1. Website Selection

The Books to Scrape practice website is selected as the data source. The website contains book-related information such as titles, prices, availability, and ratings, which are suitable for learning web scraping and data analysis.

2. Environment Setup

Google Colab is used as the development platform for writing and executing Python code. Required libraries, including Requests, BeautifulSoup, Pandas, and Matplotlib, are installed and imported.

3. Webpage Retrieval

The Requests library is used to send an HTTP request to the selected website and retrieve its HTML content. The response is checked to ensure that the webpage has been accessed successfully.

4. HTML Parsing and Data Extraction

BeautifulSoup is used to parse the HTML structure of the webpage. Relevant HTML elements are identified using CSS selectors to extract book titles, prices, availability, and ratings. The extracted information is stored in a Python list.

5. Dataset Creation

The collected book information is converted into a structured Pandas DataFrame. The dataset is organized into columns such as Title, Price, Availability, and Rating. The data is then exported as a CSV file for further analysis.

6. Data Cleaning and Preprocessing

The collected data is cleaned to improve its usability. Currency symbols are removed from price values, prices are converted into numeric format, and text-based ratings are mapped to numerical values. Duplicate records and missing values are checked and handled where necessary.

7. Data Analysis

Pandas is used to perform basic exploratory data analysis. The average, highest, and lowest book prices are calculated. Rating distributions and other relevant patterns in the dataset are also examined.

8. Data Visualization

Matplotlib is used to create visualizations such as price distribution histograms, rating distribution charts, and price comparison charts. These visualizations help present the collected information in an understandable format.

9. Results and Documentation

The analysis results, charts, Python code, and dataset are documented in the final project report. The findings are summarized based on the actual data collected and analyzed during the project.

10. Conclusion

The methodology provides a complete workflow for web scraping and data analysis, from collecting information from a website to creating a structured dataset and generating useful analytical insights.

Python Implementation – Books Web Scraping

This Python implementation collects book details from the Books to Scrape website, creates a dataset, cleans the data, performs basic analysis, and generates visualizations.

You can run this project using Google Colab or Jupyter Notebook.

1. Install Required Libraries

Run this code in Google Colab:

```
!pip install requests
beautifulsoup4 pandas matplotlib
```

2. Import Libraries

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import matplotlib.pyplot as plt
```

Explanation:

- Requests: Fetches webpage content.
- BeautifulSoup: Extracts information from HTML.
- Pandas: Creates and analyzes the dataset.
- Matplotlib: Creates charts and visualizations.

3. Select the Website

The website used for this project is:

url =

https://books.toscrape.com/

Fetch the webpage:

response = requests.get(url)

print("Status Code:", response.status_code)

Expected Output:

Status Code: 200

A status code of `200` indicates that the webpage was successfully retrieved.

4. Parse the HTML Content

soup =

BeautifulSoup(response.text, "html.parser")

print(soup.title.text)

This code converts the webpage HTML into a format that can be searched and analyzed.

5. Extract Book Details

The following code extracts the book title, price, availability, and rating.

```
books_data = []
books = soup.find_all("article", class_="product_pod")
for book in books:
    # Extract book title
    title = book.find("h3").find("a")["title"]
    # Extract book price
    price = book.find("p", class_="price_color").text.strip()
    # Extract availability
    availability = book.find(   "p", class_="instock availability" ).text.strip()
    # Extract rating
    rating_class = book.find("p", class_="star-rating")["class"]
```

```
    rating = rating_class[1]
    # Store the extracted data
    books_data.append({
        "Title": title,
        "Price": price,
        "Availability": availability,
        "Rating": rating
    })
print("Number of books extracted:", len(books_data))
```

Expected Output:

Number of books extracted: 20

The first page contains 20 books.

6. Create a Pandas DataFrame

```
df = pd.DataFrame(books_data)
df.head()
```

The dataset contains the following columns:

Column	Description
Title	Name of the book
Price	Price of the book
Availability	Stock availability
Rating	Customer rating

Display the complete dataset:

```
print(df)
```

7. Data Cleaning and Preprocessing

7.1 Clean the Price Column

The price contains the `£` symbol. Remove the symbol and convert the values into numbers.

```
df["Price"] = (
   df["Price"]
   .str.replace("£", "", regex=False)
   .astype(float)
)
df.head()
```

7.2 Convert Ratings into Numbers

The ratings are available as words such as `One`, `Two`, and `Five`.

```
    rating_mapping = {
   "One": 1,
   "Two": 2,
   "Three": 3,
   "Four": 4,
   "Five": 5
}
df["Rating"] = df["Rating"].map(rating_mapping)
df.head()
```

7.3 Clean the Availability Column

```
df["Availability"] = (
df["Availability"]
.str.replace("\n", "", regex=False)
.str.strip()
)
df.head()
```

7.4 Check Missing Values

```
print("Missing values:")
print(df.isnull().sum())
```

7.5 Remove Duplicate Records

```
df = df.drop_duplicates()
print("Dataset shape:", df.shape)
```

8. Display Dataset Information

```
print("Dataset Information:")
df.info()
```

Display statistical information:

```
print(df.describe())
```

This provides information such as:

- Number of records
- Average price
- Minimum price
- Maximum price
- Average rating

9. Data Analysis

9.1 Calculate Average Book Price

```
average_price = df["Price"].mean()
print("Average Book Price: £", round(average_price, 2))
```

9.2 Find the Most Expensive Book

```
most_expensive_book = df.loc[df["Price"].idxmax()]
print("Most Expensive Book:")
print(most_expensive_book)
```

9.3 Find the Cheapest Book

```
cheapest_book = df.loc[df["Price"].idxmin()]
print("Cheapest Book:")
print(cheapest_book)
```

9.4 Calculate Average Rating

```
average_rating = df["Rating"].mean()
print("Average Rating:", round(average_rating, 2))
```

9.5 Display Rating Distribution

```
rating_distribution = df["Rating"].value_counts().sort_index()
print("Rating Distribution:")
print(rating_distribution)
```

10. Data Visualization

10.1 Price Distribution Histogram

```
plt.figure(figsize=(8, 5))
plt.hist(df["Price"], bins=10, edgecolor="black")
plt.title("Distribution of Book Prices")
plt.xlabel("Price (£)")
plt.ylabel("Number of Books")
plt.show()
```

Purpose: Shows how book prices are distributed across the dataset.

10.2 Rating Distribution Bar Chart

```
rating_counts = df["Rating"].value_counts().sort_index()
plt.figure(figsize=(8, 5))
plt.bar(
rating_counts.index,
rating_counts.values
)
plt.title("Book Rating Distribution")
plt.xlabel("Rating")
plt.ylabel("Number of Books")
plt.xticks([1, 2, 3, 4, 5])
plt.show()
```

Purpose: Displays the number of books associated with each rating.

10.3 Top 10 Most Expensive Books

```
top_10_expensive = df.sort_values(
by="Price",
ascending=False
).head(10)
plt.figure(figsize=(12, 6))
plt.barh(
top_10_expensive["Title"],
top_10_expensive["Price"]
)
```

```
plt.title("Top 10 Most Expensive Books")
plt.xlabel("Price (£)")
plt.ylabel("Book Title")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

Purpose: Compares the prices of the ten most expensive books in the dataset.

11. Save the Dataset as a CSV File

```
df.to_csv("books_dataset.csv", index=False)
print("Dataset saved successfully!")
```

Download the CSV file in Google Colab:

```
from google.colab import files
files.download("books_dataset.csv")
```

12. Complete Python Implementation

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import matplotlib.pyplot as plt

# 1. Website URL
url = "https://books.toscrape.com/"

# 2. Fetch webpage
response = requests.get(url, timeout=10)
response.raise_for_status()
```

```
# 3. Parse HTML
 soup = BeautifulSoup(response.text, "html.parser")

# 4. Extract book details
 books_data = []
books = soup.find_all("article", class_="product_pod")
rating_mapping = {
   "One": 1,
   "Two": 2,
   "Three": 3,
   "Four": 4,
   "Five": 5
}
 for book in books:
   title = book.find("h3").find("a")["title"]
   price = book.find(
      "p", class_="price_color"
   ).text.strip()

   availability = book.find(
      "p", class_="instock availability"
   ).text.strip()

   rating_word = book.find(
      "p", class_="star-rating"
   )["class"][1]
   rating = rating_mapping.get(rating_word)
```

```
    books_data.append({
        "Title": title,
        "Price": price,
        "Availability": availability,
        "Rating": rating
    })

# 5. Create DataFrame
df = pd.DataFrame(books_data)

# 6. Clean price
df["Price"] = (
    df["Price"]
    .str.replace("£", "", regex=False)
    .astype(float)
)

# 7. Clean availability
df["Availability"] = (
    df["Availability"]
    .str.replace("\n", "", regex=False)
    .str.strip()
)

# 8. Remove duplicate records
df = df.drop_duplicates()
```

```
# 9. Display dataset
 print("Books Extracted:", len(df))
  print("\nDataset Preview:")
 print(df.head())

# 10. Perform analysis
print("\nAverage Price: £", round(df["Price"].mean(), 2))
print("Average Rating:", round(df["Rating"].mean(), 2))
print("Minimum Price: £", df["Price"].min())
print("Maximum Price: £", df["Price"].max())

# 11. Save dataset
df.to_csv("books_dataset.csv", index=False)
print("\nDataset saved as books_dataset.csv")

# 12. Price visualization
plt.figure(figsize=(8, 5))
plt.hist(df["Price"], bins=10, edgecolor="black")
plt.title("Distribution of Book Prices")
plt.xlabel("Price (£)")
plt.ylabel("Number of Books")
plt.show()

# 13. Rating visualization
rating_counts = df["Rating"].value_counts().sort_index()
plt.figure(figsize=(8, 5))
plt.bar(rating_counts.index, rating_counts.values)
```

```
plt.title("Book Rating Distribution")
plt.xlabel("Rating")
plt.ylabel("Number of Books")
plt.xticks([1, 2, 3, 4, 5])
plt.show()
```

Output:

Books Extracted: 20

Dataset Preview:

S.No	Title	Price	Availability	Rating
0	A Light in the Attic	51.77	In stock	3
1	Tipping the Velvet	53.74	In stock	1
2	Soumission	50.10	In stock	1
3	Sharp Objects	47.82	In stock	4
4	Sapiens: A Brief History of Humankind	54.23	In stock	5

Average Price: £ 38.05

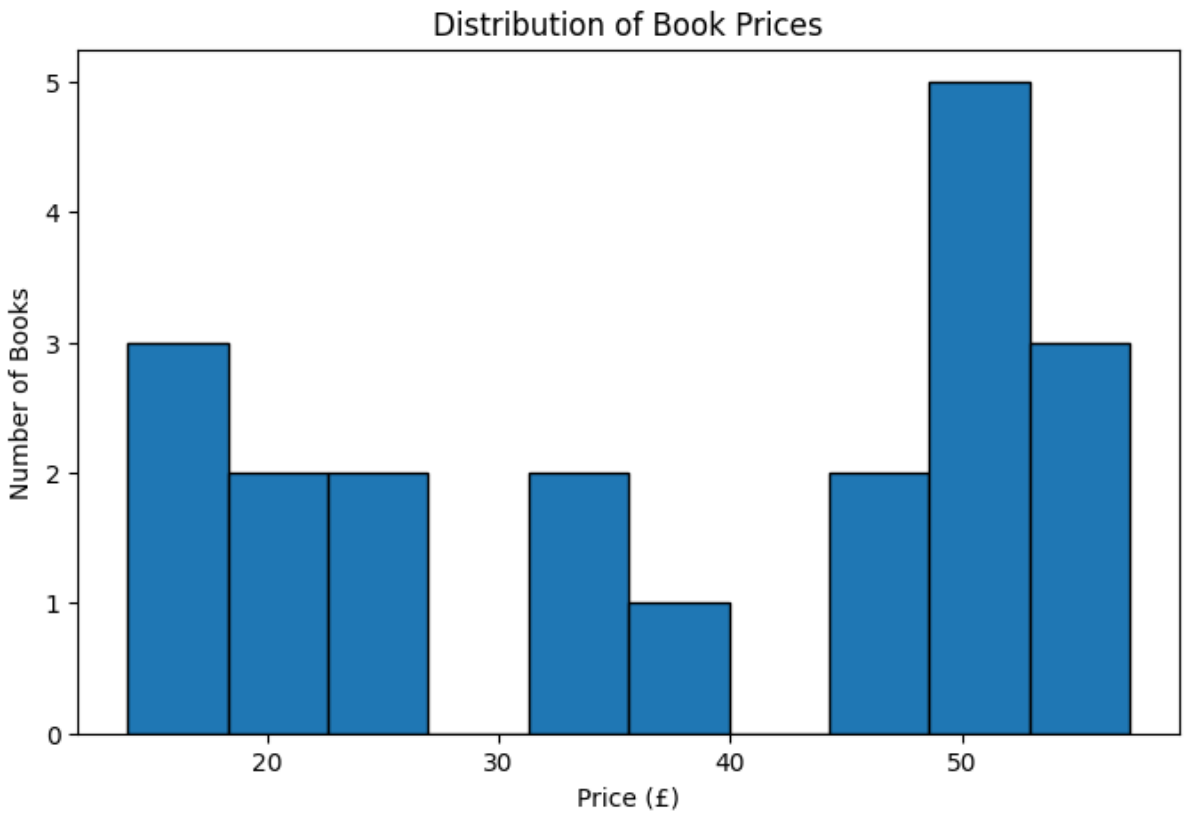
Average Rating: 2.85

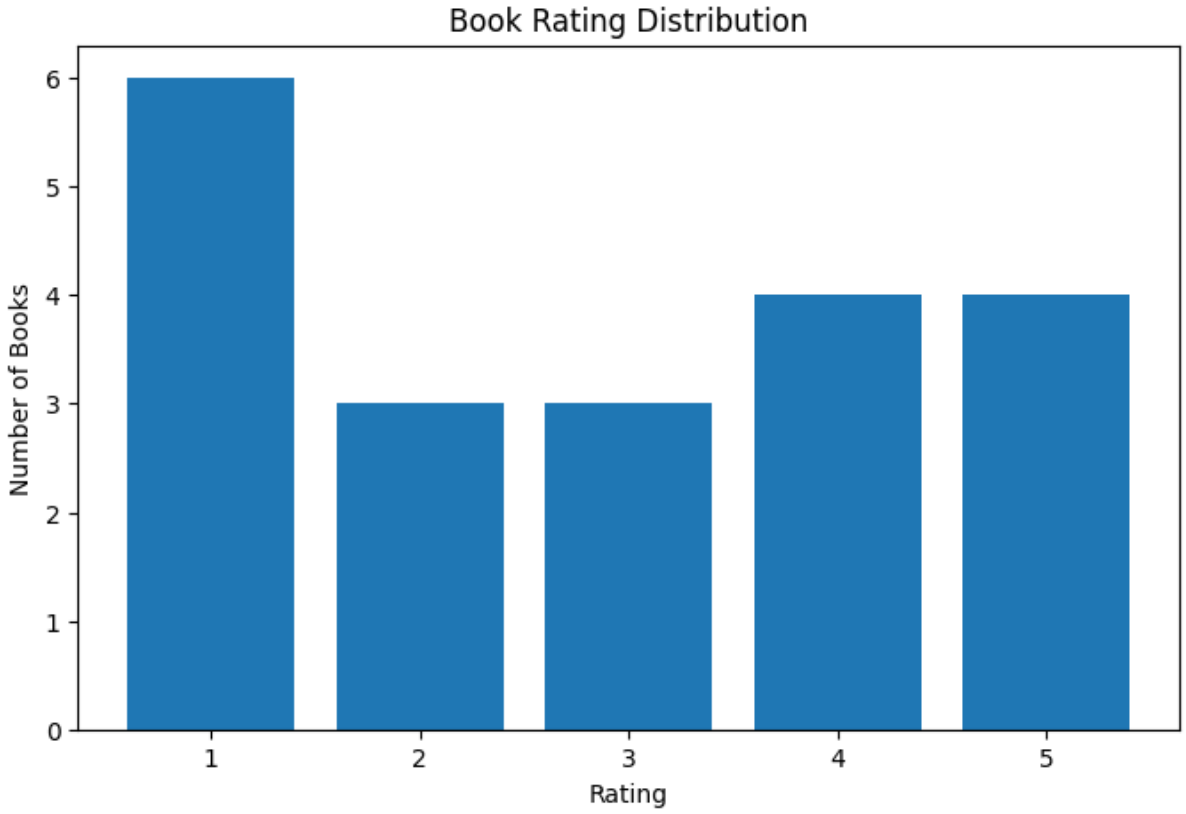
Minimum Price: £ 13.99

Maximum Price: £ 57.25

Dataset (CVS Data):

Title	Price	Availability	Rating
A Light in the Attic	51.77	In stock	3
Tipping the Velvet	53.74	In stock	1
Soumission	50.1	In stock	1
Sharp Objects	47.82	In stock	4
Sapiens: A Brief History of Humankind	54.23	In stock	5
The Requiem Red	22.65	In stock	1
The Dirty Little Secrets of Getting Your Dream Job	33.34	In stock	4
The Coming Woman: A Novel Based on the Life of the Infamous Feminist, Victoria Woodhull	17.93	In stock	3
The Boys in the Boat: Nine Americans and Their Epic Quest for Gold at the 1936 Berlin Olympics	22.6	In stock	4
The Black Maria	52.15	In stock	1
Starving Hearts (Triangular Trade Trilogy, #1)	13.99	In stock	2
Shakespeare's Sonnets	20.66	In stock	4
Set Me Free	17.46	In stock	5
Scott Pilgrim's Precious Little Life (Scott Pilgrim #1)	52.29	In stock	5
Rip it Up and Start Again	35.02	In stock	5
Our Band Could Be Your Life: Scenes from the American Indie Underground, 1981-1991	57.25	In stock	3
Olio	23.88	In stock	1
Mesaerion: The Best Science Fiction Stories 1800-1849	37.59	In stock	1
Libertarianism for Beginners	51.33	In stock	2
It's Only the Himalayas	45.17	In stock	2


Distribution of Book Prices
Number of Books
Price (£)
0
1
2
3
4
5
20
30
40
50


Book Rating Distribution
Number of Books
Rating
0
1
2
3
4
5
6
1
2
3
4
5

The following code combines the main scraping and dataset creation steps into one program.

13. Implementation Result

After executing the Python program, the following results are obtained:

- Book details are extracted from the website.
- A dataset is created using Pandas.
- Book prices are converted into numerical values.
- Ratings are converted into numbers from 1 to 5.
- Missing values and duplicate records are checked.
- Average, minimum, and maximum prices are calculated.
- Price and rating charts are generated.
- The final dataset is saved as `books_dataset.csv`.

Note: This implementation extracts the first page, which contains 20 books. The dataset size and calculated results can change if the website content changes.

Dataset and Data Cleaning – Books Web Scraping

For your Books Web Scraping project, you need to create a dataset from the extracted book details and clean it before performing data analysis.

The following content is written in a ready-to-use report format, with Python code you can execute in Google Colab.

1. Dataset Creation:

The dataset contains book information collected from the Books to Scrape website.

Dataset Attributes:

- Column Name Description Data Type
- Title Name of the book String
- Price Price of the book in pounds Float
- Availability Stock availability String
- Rating Book rating from 1 to 5 Integer
- Sample Dataset

The following is an illustrative sample of the dataset structure. The values should be replaced with the records extracted by your Python program.

- Title Price (£) Availability Rating
- A Light in the Attic 51.77 In stock 3
- Tipping the Velvet 53.74 In stock 1
- Soumission 50.10 In stock 1
- Sharp Objects 47.82 In stock 4
- Sapiens: A Brief History of Humankind 54.23 In stock 5

Important: The sample values demonstrate the dataset format. Use the CSV generated by your scraper for your actual project results.

2. Python Code for Dataset Creation

Use the following code to create a dataset from the scraped book information.

```
import pandas as pd

# Create DataFrame from scraped data

df = pd.DataFrame(books_data)


# Display the first five records

print("Dataset Preview:")

display(df.head())


# Display dataset shape

print("Dataset Shape:", df.shape)
```

Expected Output:

Dataset Shape: (20, 4)

The dataset contains 20 records and 4 columns when using the first page of the website.

3. Data Cleaning

3.1 Introduction to Data Cleaning

Data cleaning is the process of identifying and correcting errors, missing values, duplicate records, and inconsistent data formats in a dataset.

In this project, data cleaning is performed to make the scraped book data suitable for analysis and visualization.

Objectives of Data Cleaning

- Remove unwanted characters from prices.
- Convert price values into numerical format.
- Convert rating words into numbers.
- Remove unnecessary spaces and line breaks.
- Identify missing values.
- Remove duplicate records.
- Save the cleaned dataset as a CSV file.

3.2 Check the Dataset Information

```
# Display dataset information
df.info()
```

This code displays:

- Column names
- Number of records
- Data types
- Non-null values

3.3 Check Missing Values

```
# Check missing values
print("Missing Values:")
print(df.isnull().sum())
```

Explanation:

The `isnull().sum()` function counts the missing values in each column.

If the result is zero for all columns, no missing values are detected in the current dataset.

3.4 Remove Duplicate Records

```
# Check duplicate records
print("Duplicate records:", df.duplicated().sum())


# Remove duplicate records
df = df.drop_duplicates()
print("Duplicates removed successfully.")
```

Explanation:

The `drop_duplicates()` function removes repeated records from the dataset.

3.5 Clean the Price Column

The price is originally stored as a string containing the £ symbol.

Before Cleaning

£51.77
£53.74
£50.10

Python Code

```
# Remove the pound symbol and convert price to float
df["Price"] = (
  df["Price"]
   .str.replace("£", "", regex=False)
   .astype(float)
)
print(df["Price"].head())
```

After Cleaning

51.77
53.74
50.10

Result: The price column is converted from string format into a numerical format.

3.6 Clean the Availability Column

The availability column may contain unnecessary spaces and line breaks.

```
# Remove unwanted spaces and line breaks
df["Availability"] = (
 df["Availability"]
  .str.replace("\n", " ", regex=False)
   .str.strip()
 )

  print(df["Availability"].head())
```

Explanation:

- `str.replace()` removes line breaks.
- `str.strip()` removes extra spaces at the beginning and end of the text.

3.7 Convert Rating into Numerical Values

The rating is extracted as words such as `One`, `Two`, and `Five`.

```
    # Rating conversion dictionary

      rating_mapping = {

        "One": 1,

        "Two": 2,

        "Three": 3,

        "Four": 4,

         "Five": 5

      }
```

```
# Convert rating words into numbers
df["Rating"] = df["Rating"].map(rating_mapping)
print(df["Rating"].head())
```

Example:

Original Rating	Cleaned Rating
One	1
Two	2
Three	3
Four	4
Five	5

Result: Numerical ratings make it easier to calculate the average rating and create charts.

3.8 Check Invalid Values

```
# Check invalid or missing price values
print("Missing prices:", df["Price"].isnull().sum())

# Check rating values
print("Unique ratings:", df["Rating"].unique())

# Check minimum and maximum ratings
print("Minimum rating:", df["Rating"].min())
print("Maximum rating:", df["Rating"].max())
```

The rating values should be within the range of 1 to 5 after conversion.

3.9 Check Missing Values After Cleaning

```
# Display missing values after cleaning
print("Missing values after cleaning:")
print(df.isnull().sum())
```

If missing values exist, you can investigate the affected records:

```
# Display rows containing missing values
missing_rows = df[df.isnull().any(axis=1)]
display(missing_rows)
```

Do not automatically replace missing values with zero unless zero is an appropriate value for that column.

3.10 Reset the DataFrame Index

```
# Reset index after removing duplicates
df = df.reset_index(drop=True)
display(df.head())
```

This creates a new continuous index for the cleaned dataset.

4. Complete Data Cleaning Code

Copy and run the following code after extracting the book data.

```
# Import Pandas
import pandas as pd

# Create DataFrame
df = pd.DataFrame(books_data)
print("Original Dataset:")
display(df.head())
```

```
# 1. Remove duplicate records
df = df.drop_duplicates()

# 2. Clean the Price column
df["Price"] = (
    df["Price"]
    .str.replace("£", "", regex=False)
    .astype(float)
)

# 3. Clean the Availability column
df["Availability"] = (
    df["Availability"]
    .str.replace("\n", " ", regex=False)
    .str.strip()
)

# 4. Convert rating words into numbers
rating_mapping = {
    "One": 1,
    "Two": 2,
    "Three": 3,
    "Four": 4,
    "Five": 5
}
df["Rating"] = df["Rating"].map(rating_mapping)
```

```
# 5. Reset index
df = df.reset_index(drop=True)

# 6. Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# 7. Display data types
print("\nData Types:")
print(df.dtypes)

# 8. Display cleaned dataset
print("\nCleaned Dataset:")
display(df.head())

# 9. Display dataset shape
print("Final Dataset Shape:", df.shape)

# 10. Save cleaned dataset
df.to_csv("cleaned_books_dataset.csv", index=False)

print("\nCleaned dataset saved successfully!")
```

5. Before and After Data Cleaning

Data Item	Before Cleaning	After Cleaning
Price	£51.77	51.77
Price Data Type	String	Float
Rating	Five	5
Rating Data Type	String	Integer
Availability	Text with spaces /line breaks	Clean text
Duplicate Records	May be present	Removed
Index	Original index	Reset index

6. Save and Download the Cleaned Dataset

Save as CSV

```
df.to_csv("cleaned_books_dataset.csv", index=False)
```

Download in Google Colab

```
from google.colab import files

files.download("cleaned_books_dataset.csv")
```

The cleaned dataset can be used for your next project section: Data Analysis and Visualization.

7. Ready-to-Submit Report Content

Dataset and Data Cleaning

Dataset:

The dataset for this project was created by extracting book details from the Books to Scrape website using Python, Requests, and BeautifulSoup. The collected dataset contains information such as book title, price, availability, and rating. Pandas was used to organize the extracted information into a structured DataFrame.

The dataset was saved in CSV format for further processing and analysis.

Data Cleaning:

Data cleaning was performed to improve the quality and consistency of the scraped dataset. The price column was cleaned by removing the pound symbol and converting the values into floating-point numbers. The rating values, which were originally represented as words, were converted into numerical values ranging from 1 to 5.

Unnecessary spaces and line breaks were removed from the availability column. Duplicate records were checked and removed, and missing values were identified using Pandas. Finally, the DataFrame index was reset, and the cleaned dataset was saved as a CSV file.

Result:

After data cleaning, the dataset was organized into a suitable format for data analysis and visualization. The cleaned data can be used to calculate average prices, analyze rating distributions, and create charts using Matplotlib.

Next project section:

Data Analysis and Visualization, including average price, highest-priced book, rating distribution, and charts.

Data Analysis and Visualization – Books Web Scraping

1. Introduction:

Data analysis and visualization are important steps in the Books Web Scraping project. After collecting and cleaning the book dataset, Python libraries such as Pandas and Matplotlib are used to analyze book prices, ratings, and availability.

The main purpose of this stage is to identify useful patterns in the dataset and present the results using charts and statistical values.

2. Objectives:

- Calculate the average book price.
- Identify the cheapest and most expensive books.
- Analyze the distribution of book ratings.
- Count the availability of books.
- Create graphs to visualize the collected data.
- Identify useful patterns from the dataset.

3. Import Required Libraries:

Run the following code in Google Colab:

```
import pandas as pd

import matplotlib.pyplot as plt
```

Load your cleaned dataset:

```
df = pd.read_csv("cleaned_books_dataset.csv")

print("Dataset loaded successfully!")

display(df.head())
```

Note: If your DataFrame `df` is already available, you can continue without loading the CSV file again.

4. Descriptive Data Analysis:

4.1 Display Dataset Information

```
print("Dataset Information:")
df.info()
```

This displays the column names, data types, and number of non-null values.

4.2 Calculate Average Book Price

```
average_price = df["Price"].mean()
print("Average Book Price: £", round(average_price, 2))
```

Explanation:

The `mean()` function calculates the average price of all books in the dataset.

Report interpretation:

The average book price represents the typical price of books collected from the website.

4.3 Find the Cheapest Book

```
cheapest_book = df.loc[df["Price"].idxmin()]
print("Cheapest Book:")
print(cheapest_book)
```

Explanation:

The `idxmin()` function identifies the row containing the minimum price.

4.4 Find the Most Expensive Book

```
most_expensive_book = df.loc[df["Price"].idxmax()]
print("Most Expensive Book:")
print(most_expensive_book)
```

Explanation:

The `idxmax()` function identifies the row containing the maximum price.

4.5 Calculate Average Rating

```
average_rating = df["Rating"].mean()
print("Average Book Rating:", round(average_rating, 2))
```

Explanation:

The average rating is calculated using the numerical rating values from 1 to 5.

4.6 Display Statistical Summary

```
print("Statistical Summary:")
display(df[["Price", "Rating"]].describe())
```

This provides:

Statistical Value	Description
Count	Number of records
Mean	Average value
Std	Standard deviation
Min	Minimum value
25%	First quartile
50%	Median
75%	Third quartile
Max	Maximum value

5. Data Visualization:

5.1 Histogram – Distribution of Book Prices

A histogram shows how book prices are distributed.

```
plt.figure(figsize=(8, 5))


plt.hist(
 df["Price"],
 bins=10,
  edgecolor="black"
)


plt.title("Distribution of Book Prices")
plt.xlabel("Price (£)")
 plt.ylabel("Number of Books")


 plt.tight_layout()
  plt.show()
```

Interpretation:

The histogram displays the number of books within different price ranges. It helps identify whether the books are concentrated in a particular price range.

5.2 Bar Chart – Rating Distribution

This chart displays the number of books for each rating.

```
rating_counts = df["Rating"].value_counts().sort_index()


 plt.figure(figsize=(8, 5))
```

```
plt.bar(
 rating_counts.index,
 rating_counts.values
)

 plt.title("Book Rating Distribution")
 plt.xlabel("Rating")
 plt.ylabel("Number of Books")
 plt.xticks([1, 2, 3, 4, 5])

 plt.tight_layout()
 plt.show()
```

Interpretation:

The bar chart helps identify which ratings occur most frequently in the dataset.

5.3 Bar Chart – Top 10 Most Expensive Books

```
top_10_expensive = df.sort_values(
    by="Price",
    ascending=False
 ).head(10)

plt.figure(figsize=(10, 6))

plt.barh(
    top_10_expensive["Title"],
    top_10_expensive["Price"]
 )
```

```
plt.title("Top 10 Most Expensive Books")
plt.xlabel("Price (£)")
plt.ylabel("Book Title")


plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

Interpretation:

This chart compares the prices of the ten most expensive books in the scraped dataset.

5.4 Availability Analysis:

First, count the different availability values:

```
availability_counts = df["Availability"].value_counts()
print("Availability Distribution:")
print(availability_counts)
```

Create a bar chart:

```
plt.figure(figsize=(8, 5))

plt.bar(
    availability_counts.index,
    availability_counts.values
)

plt.title("Book Availability Distribution")
plt.xlabel("Availability")
plt.ylabel("Number of Books")

plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

Interpretation:

The chart shows the number of books associated with each availability status. The first page of the practice website commonly contains books marked as available in stock.

5.5 Scatter Plot – Price vs Rating:

A scatter plot can be used to explore the relationship between book price and rating.

```
plt.figure(figsize=(8, 5))

plt.scatter(
    df["Rating"],
    df["Price"]
)

plt.title("Relationship Between Book Rating and Price")
plt.xlabel("Rating")
plt.ylabel("Price (£)")

plt.xticks([1, 2, 3, 4, 5])
plt.tight_layout()
plt.show()
```

Interpretation:

The scatter plot shows how book prices are distributed across different ratings. It can help identify possible patterns, but it does not establish that ratings cause higher or lower prices.

6. Complete Data Analysis Code

The following code performs the main analysis and generates the visualizations.

```
import pandas as pd

import matplotlib.pyplot as plt
```

```
# 1. Load cleaned dataset
df = pd.read_csv("cleaned_books_dataset.csv")

# 2. Display dataset
print("Dataset Preview:")
display(df.head())

# 3. Basic analysis
average_price = df["Price"].mean()
average_rating = df["Rating"].mean()
minimum_price = df["Price"].min()
maximum_price = df["Price"].max()
print("\nDATA ANALYSIS RESULTS")
print("-" * 30)
print("Total Books:", len(df))
print("Average Price: £", round(average_price, 2))
print("Average Rating:", round(average_rating, 2))
print("Minimum Price: £", minimum_price)
print("Maximum Price: £", maximum_price)

# 4. Cheapest book
print("\nCHEAPEST BOOK")
print(df.loc[df["Price"].idxmin()])

# 5. Most expensive book
print("\nMOST EXPENSIVE BOOK")
print(df.loc[df["Price"].idxmax()])
```

```
# 6. Statistical summary
print("\nSTATISTICAL SUMMARY")
display(df[["Price", "Rating"]].describe())

# 7. Price histogram
plt.figure(figsize=(8, 5))
plt.hist(df["Price"], bins=10, edgecolor="black")
plt.title("Distribution of Book Prices")
plt.xlabel("Price (£)")
plt.ylabel("Number of Books")
plt.tight_layout()
plt.show()

# 8. Rating bar chart
rating_counts = df["Rating"].value_counts().sort_index()

plt.figure(figsize=(8, 5))
plt.bar(rating_counts.index, rating_counts.values)
plt.title("Book Rating Distribution")
plt.xlabel("Rating")
plt.ylabel("Number of Books")
plt.xticks([1, 2, 3, 4, 5])
plt.tight_layout()
plt.show()

# 9. Top 10 expensive books
top_10_expensive = df.sort_values(
```

```
 by="Price",  ascending=False).head(10)
plt.figure(figsize=(10, 6))
plt.barh(
   top_10_expensive["Title"],
   top_10_expensive["Price"]
)
plt.title("Top 10 Most Expensive Books")
plt.xlabel("Price (£)")
plt.ylabel("Book Title")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

# 10. Availability chart
availability_counts = df["Availability"].value_counts()

plt.figure(figsize=(8, 5))
plt.bar(
   availability_counts.index,
   availability_counts.values
)
plt.title("Book Availability Distribution")
plt.xlabel("Availability")
plt.ylabel("Number of Books")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

```
# 11. Price vs rating scatter plot
plt.figure(figsize=(8, 5))
plt.scatter(df["Rating"], df["Price"])
plt.title("Relationship Between Book Rating and Price")
plt.xlabel("Rating")
plt.ylabel("Price (£)")
plt.xticks([1, 2, 3, 4, 5])
plt.tight_layout()
plt.show()
```

7. Sample Analysis Results Format

After running the code, copy your actual output into the following table.

Analysis	Result
Total number of books	20
Average book price	£ 38.05
Minimum book price	£ 13.99
Maximum book price	£ 57.25
Average rating	2.85

Results and Conclusion – Books Web Scraping

1. Results:

The Books Web Scraping project was implemented using Python libraries such as Requests, BeautifulSoup, Pandas, and Matplotlib. Book information was collected from the Books to Scrape website and organized into a structured dataset.

Results Obtained:

- Book details such as title, price, availability, and rating were extracted successfully.
- The extracted data was converted into a Pandas DataFrame.
- Unwanted characters, including the pound symbol in the price column, were removed during data cleaning.
- Book prices were converted into numerical values for analysis.
- Rating values were converted from words into numerical values ranging from 1 to 5.
- Missing values and duplicate records were checked.
- Statistical analysis was performed to calculate the average, minimum, and maximum book prices.
- Visualizations such as histograms, bar charts, and scatter plots were created using Matplotlib.
- The cleaned dataset was saved in CSV format for future use.

Results Summary

Analysis	Result
Data source	Books to Scrape website
Scraping tool	Python and BeautifulSoup
Dataset library	Pandas

Visualization library	Matplotlib
Data collected	Title, Price, Availability, Rating
Data cleaning	Completed using Pandas
Output format	CSV
Visualizations	Histogram, bar chart, scatter plot

Note: The actual average price, minimum price, maximum price, and rating values should be copied from your Python program's output. The numerical results may vary depending on the records collected.

2. Result Interpretation

2.1 Book Price Analysis

The book prices were analyzed using descriptive statistical methods. The average price was calculated, and the cheapest and most expensive books were identified.

The price histogram helped display the distribution of book prices across different price ranges.

2.2 Rating Analysis

The rating values were converted into numbers from 1 to 5. A bar chart was created to show the frequency of each rating in the dataset.

2.3 Availability Analysis

The availability column was analyzed to identify the stock status of the collected books. The results were displayed using a bar chart.

2.4 Data Visualization

Matplotlib was used to create different charts, including:

- Book price distribution histogram
- Book rating distribution bar chart
- Top 10 most expensive books chart
- Availability distribution chart

- Price versus rating scatter plot

These visualizations made the data easier to understand and compare.

3. Conclusion

The Books Web Scraping project demonstrated the complete process of collecting, cleaning, analyzing, and visualizing data from a website using Python.

BeautifulSoup was used to extract book information from HTML content, while Pandas helped organize and clean the dataset. Matplotlib was used to create charts that represented book prices, ratings, and availability.

This project improved practical knowledge of web scraping, data preprocessing, exploratory data analysis, and data visualization. The collected dataset can be used for further analysis and can be expanded by scraping additional pages from the website.

The project also provided an understanding of how raw website information can be transformed into a structured dataset suitable for data analysis.

4. Future Enhancements

- Scrape all available pages instead of only the first page.
- Collect additional information such as product categories and book URLs.
- Analyze the relationship between book prices and ratings.
- Create an interactive dashboard using Power BI or Streamlit.
- Schedule the scraper to collect updated data periodically.
- Export the analyzed results into Excel or other data formats.

Dataset (CSV Data):

Title	Price	Availability	Rating
A Light in the Attic	51.77	In stock	3
Tipping the Velvet	53.74	In stock	1
Soumission	50.1	In stock	1
Sharp Objects	47.82	In stock	4
Sapiens: A Brief History of Humankind	54.23	In stock	5
The Requiem Red	22.65	In stock	1
The Dirty Little Secrets of Getting Your Dream Job	33.34	In stock	4
The Coming Woman: A Novel Based on the Life of the Infamous Feminist, Victoria Woodhull	17.93	In stock	3
The Boys in the Boat: Nine Americans and Their Epic Quest for Gold at the 1936 Berlin Olympics	22.6	In stock	4
The Black Maria	52.15	In stock	1
Starving Hearts (Triangular Trade Trilogy, #1)	13.99	In stock	2
Shakespeare's Sonnets	20.66	In stock	4
Set Me Free	17.46	In stock	5
Scott Pilgrim's Precious Little Life (Scott Pilgrim #1)	52.29	In stock	5
Rip it Up and Start Again	35.02	In stock	5
Our Band Could Be Your Life: Scenes from the American Indie Underground, 1981-1991	57.25	In stock	3
Olio	23.88	In stock	1
Mesaerion: The Best Science Fiction Stories 1800-1849	37.59	In stock	1
Libertarianism for Beginners	51.33	In stock	2
It's Only the Himalayas	45.17	In stock	2

1. References

You can copy the following references directly into your project report.

2. References with Their Uses

Reference	Use in your project
Books to Scrape	Website used for collecting book data
Requests	Retrieving webpage content
BeautifulSoup	Parsing HTML and extracting book details
Pandas	Dataset creation, cleaning, and analysis
Matplotlib	Creating graphs and charts
Python Documentation	Understanding Python programming
Google Colab	Running Python code in the cloud

The Beautiful Soup documentation explains HTML parsing and methods such as `find()` and `find_all()`. Pandas documentation provides guidance on data processing and input/output operations, while Matplotlib provides resources for Python-based visualization.